

LLM-Aided Testbench Generation Report

1. Overview

In this homework I used the provided Enhanced_LLM_Aided_Testbench_Generation notebook to generate self-checking Verilog testbenches. I ran two tutorial examples first, then I made one small custom module. I also intentionally added one bug to the custom module to check if the generated testbench can catch it.

The two tutorial examples I used were the 2-to-1 MUX and the 4-bit adder. My custom module was a 4-bit saturating adder.

2. LLM Setup

I ran the notebook in Google Colab. The LLM provider was OpenAI and the model used in the notebook was gpt-4o. I did not make special changes to temperature or other model parameters. Most prompts were already in the notebook. For my custom module, I wrote a simple natural language description and asked for at least 20 test patterns with some corner cases and random cases.

I stored the API key in Colab Secrets and loaded it into OPENAI_API_KEY. I did not put the real API key in the notebook or GitHub.

3. Tutorial Example 1 - 2-to-1 MUX

The first example was a simple 2-to-1 multiplexer. It has three 1-bit inputs a, b, and sel, and one 1-bit output y. When sel is 0, y should be a. When sel is 1, y should be b.

MUX interface

Port	Direction	Width
a	input	1 bit
b	input	1 bit
sel	input	1 bit
y	output	1 bit

The pipeline generated testbench_initial.v, golden_model.py, test_patterns_with_golden.json, and testbench_final.v. The final testbench was compiled with Icarus Verilog and then executed with vvp.

```
iverilog -g2012 -o LLM-aided-Testbench-Generation/examples/output/mux/sim.vvp LLM-aided-Testbench-Generation/examples/output/mux2to1.v LLM-aided-Testbench-Generation/examples/output/mux/testbench_final.v
vvp LLM-aided-Testbench-Generation/examples/output/mux/sim.vvp
```

The simulation finished successfully. There were 8 tests total, 8 passed, and 0 failed.

```
Total tests run: 8
Number passed: 8
Number failed: 0
```

4. Tutorial Example 2 - 4-bit Adder

The second example was a 4-bit unsigned adder. It has two 4-bit inputs a and b. The lower 4 bits of the addition result are connected to sum, and the extra carry bit is connected to carry.

4-bit adder interface

Port	Direction	Width
a	input	4 bits
b	input	4 bits
sum	output	4 bits
carry	output	1 bit

The Verilog code used a 5-bit internal result so that the carry bit was not lost. I ran the same generation flow and then compiled and simulated the generated final testbench. The simulation passed all generated tests.

```
iverilog -g2012 -o LLM-aided-Testbench-Generation/examples/output/adder/sim.vvp LLM-aided-Testbench-Generation/examples/output/adder4bit.v LLM-aided-Testbench-Generation/examples/output/adder/testbench_final.v
vvp LLM-aided-Testbench-Generation/examples/output/adder/sim.vvp
```

5. Generated Artifact Explanation

I inspected the generated files using the MUX example.

golden_model.py: This file is a Python reference model. For the MUX, it takes a, b, and sel as inputs. It returns $y = a$ when sel is 0, otherwise it returns $y = b$. I used this as the correct reference output.

test_patterns_with_golden.json: This file contains the test input patterns and the expected output for each pattern. The expected values are calculated by the Python golden model.

Difference between initial and final testbench: testbench_initial.v mainly contains the DUT setup and input test patterns. testbench_final.v has the expected values added and also includes compare/check logic. It prints pass or fail information automatically, so I do not need to check every output by hand.

6. Custom Module - 4-bit Saturating Adder

For my custom module I made a 4-bit unsigned saturating adder called saturating_adder4. It has two 4-bit inputs a and b and one 4-bit output y. If $a + b$ is 15 or smaller, y is the normal sum. If $a + b$ is bigger than 15, y stays at 15.

Custom module interface

Port	Direction	Width
a	input	4 bits
b	input	4 bits
y	output	4 bits

I asked the LLM pipeline to generate at least 20 patterns including corner cases and random cases. The pipeline generated 20 patterns and computed golden outputs for all 20 patterns. Some corner cases included $0 + 0$, $15 + 0$, and $15 + 15$.

```
iverilog -g2012 -o LLM-aided-Testbench-Generation/examples/output/custom/sim.vvp LLM-aided-Testbench-Generation/examples/output/custom/saturating_adder4.v LLM-aided-Testbench-Generation/examples/output/custom/testbench_final.v
vvp LLM-aided-Testbench-Generation/examples/output/custom/sim.vvp
```

The correct custom module passed all generated tests.

Test Summary: Total Tests = 20, Passed = 20, Failed = 0

7. Bug Detection Test

To test if the self-checking testbench can really find a bug, I intentionally changed the saturation value. The correct behavior should output 15 when the sum is larger than 15, but I changed the buggy version to output 14 instead.

```
Correct: assign y = (sum_ext > 5'd15) ? 4'd15 : sum_ext[3:0];  
Buggy:  assign y = (sum_ext > 5'd15) ? 4'd14 : sum_ext[3:0];
```

After I ran the same final testbench with the buggy DUT, the testbench detected the wrong output. For example, one failing line was:

```
Test 10 Failed: Expected y = 1111, Got y = 1110
```

The overall result with the bug was:

```
Test Summary: Total Tests = 20, Passed = 17, Failed = 3
```

Then I changed the DUT back to the correct saturation value 15 and ran the test again. After the fix, all 20 tests passed again.

```
Test Summary: Total Tests = 20, Passed = 20, Failed = 0
```

8. Conclusion

This homework helped me understand how an LLM can be used to create test patterns and a self-checking Verilog testbench. The Python golden model was useful because it gave the expected output automatically. I also saw that the generated testbench was able to catch an intentional implementation bug. The MUX, 4-bit adder, and my saturating adder all worked after simulation and verification.